

Увод в програмирането - Указатели, масиви, рекурсия

Ивайло Андреев + gemeni 3 Flash, chat gpt 5.4

16 май 2026

Съдържание

1	Указатели и указателна аритметика	2
1.1	Основни операции и аритметика	2
2	Едномерни и многомерни масиви	2
2.1	Индексиране и предаване на масиви	2
2.2	Основни алгоритми за масиви (Текст и код)	3
3	Символни низове	4
3.1	Основни операции	5
4	Рекурсия	5
4.1	Етапи на изпълнение и механизъм	5
4.2	Класификация на видовете рекурсия (Текст и код)	6

1 Указатели и указателна аритметика

Указателят е специален вид променлива, която съхранява като своя стойност **адрес на друга променлива** в паметта. Вместо да пази директни данни, той сочи към местоположението им. Размерът на указателя е фиксиран и зависи единствено от архитектурата на процесора (4 байта при 32-битови системи, 8 байта при 64-битови системи), независимо от типа данни, към който сочи.

1.1 Основни операции и аритметика

Две основни операции дефинират работата с указатели: вземане на адрес (&) и дерефеждане (*).

```
1 int a = 5;
2 int *p = &a; // '&' takes the address of variable 'a'
3 int value = *p; // '*' dereferences the pointer to get the value (5)
```

Указателна аритметика: Към указател може да се добавя или изважда цяло число. При добавяне на 1, адресът не се увеличава с 1 байт, а се измества с размера на типа данни, към който сочи: $\text{нов_адрес} = \text{текущ_адрес} + i \times \text{sizeof}(T)$. Две указателя от един и същи тип могат да се изваждат, което връща броя елементи между тях.

2 Едномерни и многомерни масиви

Масивът е структура от данни, състояща се от фиксиран брой елементи от **един и същи тип**, които са разположени последователно в **непрекъснат блок от паметта**. Името на масива не е указател, но в много изрази се преобразува до адреса на неговия първи елемент (индекс 0).

2.1 Индексиране и предаване на масиви

Операцията индексиране осигурява директен достъп до елементите. Поради непрекъснатостта на паметта, изразът `arr[i]` е напълно еквивалентен на указателния запис `*(arr + i)`.

Многомерните масиви (напр. двумерни) се разполагат в паметта линейно, ред по ред (*Row-major order*). За да може компилаторът да пресметне точния адрес на даден елемент `matrix[i][j]`, той трябва да знае размера на редовете. Затова при предаване на многомерен масив като параметър във функция, **задължително трябва да се указват размерите на всички измерения без първото**.

Понеже елементите на всеки ред лежат последователно в паметта, на практика е по-ефективно масивът да се обхожда **по редове, а не по колони**. Така достъпът е по-добър за кеша на процесора (*cache locality*) и програмата обикновено работи по-бързо.

Пример за функция, която приема двумерен масив с 3 колони:

```

1 #include <iostream>
2
3 void printMatrix(int matrix[][3], int rows) {
4     for (int i = 0; i < rows; i++) {
5         for (int j = 0; j < 3; j++) {
6             std::cout << matrix[i][j] << " ";
7         }
8         std::cout << std::endl;
9     }
10 }
11
12 int main() {
13     int data[2][3] = {
14         {1, 2, 3},
15         {4, 5, 6}
16     };
17
18     printMatrix(data, 2);
19     return 0;
20 }

```

Тук `matrix[][3]` показва, че броят на редовете може да се подаде отделно, но броят на колоните трябва да е известен в сигнатурата на функцията.

2.2 Основни алгоритми за масиви (Текст и код)

В съответствие с изискването за смесване на обяснения и програмни блокове, тук са разписани четирите фундаментални алгоритъма:

1. Bubble Sort (Сортиране чрез мехурчето): Алгоритъмът обхожда масива многократно, като сравнява всеки два съседни елемента и ги разменя, ако са в грешен ред. По този начин най-големият елемент постепенно "изплува" към края. Сложност: $O(n^2)$.

```

1 void bubbleSort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         for (int j = 0; j < n - i - 1; j++) {
4             if (arr[j] > arr[j + 1]) {
5                 // Swap adjacent elements if left is greater than right
6                 int temp = arr[j];
7                 arr[j] = arr[j + 1];
8                 arr[j + 1] = temp;
9             }
10        }
11    }
12 }

```

2. Selection Sort (Сортиране чрез избор): Алгоритъмът разделя масива на сортирана и несортирана част. При всяка стъпка намира най-малкия елемент от несортираната част и го разменя с първия елемент от нея. Сложност: $O(n^2)$.

```

1 void selectionSort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int minIdx = i; // Track the minimum index in unsorted part

```

```

4     for (int j = i + 1; j < n; j++) {
5         if (arr[j] < arr[minIdx]) {
6             minIdx = j; // Found a smaller element, update index
7         }
8     }
9     // Swap the found minimum with the first unsorted position
10    int temp = arr[i];
11    arr[i] = arr[minIdx];
12    arr[minIdx] = temp;
13 }
14 }

```

3. Linear Search (Последователно търсене): Обхожда елементите един по един от началото до края, докато намери търсената стойност. Работи на всякакви масиви (сортирани и несортирани). Сложност: $O(n)$.

```

1 int linearSearch(int arr[], int n, int target) {
2     for (int i = 0; i < n; i++) {
3         if (arr[i] == target) {
4             return i; // Target found, return current index
5         }
6     }
7     return -1; // Target not found in the array
8 }

```

4. Binary Search (Двоично търсене): Изключително ефективен алгоритъм, но работи само върху предварително сортирани масиви. Разделя интервала за търсене наполовина при всяка стъпка, сравнявайки средния елемент с търсената стойност. Сложност: $O(\log n)$.

```

1 int binarySearch(int arr[], int n, int target) {
2     int left = 0, right = n - 1;
3     while (left <= right) {
4         int mid = left + (right - left) / 2; // Avoid potential overflow
5         if (arr[mid] == target) {
6             return mid; // Target element located
7         }
8         if (arr[mid] < target) {
9             left = mid + 1; // Discard left half
10        } else {
11            right = mid - 1; // Discard right half
12        }
13    }
14    return -1; // Target not found
15 }

```

3 Символни низове

Традиционните C-style низове се представят в паметта като **едномерен масив от символи (char)**, който завършва със специален терминаращ символ – **нулев байт '\0'** (ASCII код 0). Този символ указва края на низа в непрекъснатата па-

мет. Всеки символ заема точно 1 байт. Например, низът "hello" заема 6 байта в паметта заради знака '\0' накрая.

3.1 Основни операции

Управлението на C-style низове става чрез стандартни функции от библиотеката <cstring>. Ето как се използват те в практически код:

```
1 #include <cstring>
2
3 void stringDemo() {
4     char h[] = "hello"; // same as char h[] = {'h', 'e', 'l', 'l', 'o',
5     '\0'};
6     char buffer[20];
7
8     // 1. strlen: Returns the number of characters excluding '\0'
9     int len = strlen(h);
10
11    // 2. strcpy: Copies the source string content into the destination
12    buffer
13    strcpy(buffer, h);
14
15    // 3. strcat: Appends (concatenates) source text to the destination
16    strcat(buffer, " world");
17
18    // 4. strcmp: Lexicographically compares two strings (returns 0 if
19    equal)
20    int cmp = strcmp(buffer, h);
21 }
```

В съвременното C++ често се предпочита `std::string` от библиотеката <string>, защото е *high-level* абстракция за работа с низове. Той управлява автоматично паметта, знае дължината си и намалява риска от често срещани грешки при C-style низовете, като излизане извън границите на масива или липсващ терминиращ символ '\0'. Затова на практика `std::string` е препоръчителният избор, а C-style низовете се използват главно при ниско ниво на работа, стари библиотеки или специфични задачи.

4 Рекурсия

Рекурсията е програмен подход, при който една функция се извиква самата себе си (директно или индиректно) за решаване на подпроблем.

4.1 Етапи на изпълнение и механизъм

Изпълнението на рекурсията се поддържа от механизма на извикванията на функции и системния стек (**Call Stack**). При всяко извикване се заделя **стекова рамка (Stack Frame)**, съдържаща локалните променливи и адреса за връщане. Механизмът може да се разгледа в два основни етапа:

- **Разгъване (Expansion):** Функцията се вика многократно, създавайки нови стекови рамки нагоре в паметта. Максималният брой вложени извиквания дефинира *дълбочината на рекурсията*.
- **Свиване (Collapse):** След достигане на **базовия случай (дъно)**, рекурсията спира да се разклонява и започва връщането на резултати обратно към предходните извиквания, като стековите рамки се разрушават една по една.

Липсата на базов случай води до безкрайна рекурсия и препълване на стека (*stack overflow*). В много случаи рекурсивен алгоритъм може да бъде реализиран и итеративно, като извикванията се симулират чрез структурата от данни стек.

4.2 Класификация на видовете рекурсия (Текст и код)

1. Пряка (Директна) рекурсия: Функция директно извиква себе си в своето тяло.

```
1 void directRec(int n) {
2     if (n <= 0) return; // Base case
3     directRec(n - 1); // Function calls itself directly
4 }
```

2. Косвена (Индиректна) рекурсия: Функция А извиква функция В, а функция В от своя страна извиква функция А.

```
1 void functionB(int n); // Forward declaration
2
3 void functionA(int n) {
4     if (n <= 0) return; // Base case
5     functionB(n - 1); // Call function B
6 }
7
8 void functionB(int n) {
9     if (n <= 0) return; // Base case
10    functionA(n - 1); // Mutual call back to function A
11 }
```

3. Линейна рекурсия: При всяко изпълнение функцията генерира най-много **едно** рекурсивно извикване (пример: пресмятане на Факториел $n!$).

```
1 unsigned long long factorial(int n) {
2     if (n <= 1) return 1; // Base case
3     return n * factorial(n - 1); // Exactly one recursive call per step
4 }
```

4. Разклонена рекурсия: Тялото на функцията съдържа **две или повече** рекурсивни извиквания, генерирайки дървовидна структура от извиквания (пример: Числа на Фибоначи).

```
1 int fibonacci(int n) {
2     if (n <= 0) return 0; // Base case 1
3     if (n == 1) return 1; // Base case 2
4     // Multiple branching recursive calls
5 }
```

```
5     return fibonacci(n - 1) + fibonacci(n - 2);
6 }
```

***бонус - Рекурсивно двоично търсене:** Двоичното търсене може да бъде реализирано и рекурсивно. При всяко извикване алгоритъмът сравнява средния елемент с търсената стойност и рекурсивно продължава само в едната половина на масива. Алгоритъмът работи само върху сортирани масиви и има логаритмична сложност $O(\log n)$.

```
1 int recursiveBinarySearch(int arr[], int left, int right, int target) {
2     if (left > right) {
3         return -1; // Target not found
4     }
5
6     int mid = left + (right - left) / 2;
7
8     if (arr[mid] == target) {
9         return mid; // Target found
10    }
11
12    if (arr[mid] > target) {
13        return recursiveBinarySearch(arr, left, mid - 1, target);
14    }
15
16    return recursiveBinarySearch(arr, mid + 1, right, target);
17 }
```